

hrace_driver.py — Full Function-Suite Demonstration

Driver: /home/mrnob0dy666/imsqct/Vox/hrace_driver.py (429 lines, stdlib-only, self-contained)

Demonstration: 19 live runs (16 CLI-surface + 3 follow-up), executed 2026-02-14.

Raw transcripts: /tmp/hrace_results.txt (runs 01-16), /tmp/hrace_results2.txt (runs 17-19).

Reproduce any run with the exact command quoted under it.

1 What the driver is

An HIV gp120 C4 (V3 loop) antibody-escape “arms race” pipeline. Given a wild-type epitope and three CDR contact sets, it:

1. enumerates every single-amino-acid mutant (the *landscape*),
2. scores each mutant’s **gap** (escape severity 2/3/4) using a **glyph-routed rule engine** — the 12-primitive Shavian glyph table (Red-Hot Rebis / SNS_PRIME) is the rule engine, not a physicochemical proxy,
3. flags escape threats ($\text{gap} \geq 3$, `frob=True`),
4. picks the TOP ESCAPE and renders it with a caret,
5. judges whether a counter-CDR3 *engages* the CDR3 via 7-bottleneck glyph agreement,
6. builds the vector DNA (CDR3 + linker + counter) and verifies translation round-trip,
7. measures CDR1/CDR2 coverage of the threat set,
8. prints FROBENIUS: all checks closed (B4=T) when its internal asserts hold.

Indexing is 0-based throughout; the kernel’s “position 15” escape is WT[15] == 'I'.

1.1 Glyph algebra (the rule engine)

Each amino acid carries a 12-tuple $\langle \vdash \neg \succ \prec \bowtie \top \in \exists \odot \perp \boxplus \boxdot \rangle$ of Shavian letters (SIDECHAINS, lines 102-123). Two operations over the tuples:

- `glyph_distance(a, b)` — Euclidean distance on primitive ordinals (PRIM_ORD).
- `bottleneck_count(a, b)` — disagreements on the 7 SNS_PRIME bottleneck slots ($\succ \prec \top \odot \perp \boxplus \boxdot$).

1.2 Gap models

- **pinned** (canonical only): position class from the kernel-verified CORE4/TIER3/BASE partition. CORE4 pins $\{(15,D), (7,E), (21,K), (2,D), (26,R), (18,E)\}$ get gap 4; other CORE4 get 3; TIER3 gets 3 if the mutant is a charged glyph (D/E/K/R) else 2; BASE gets 2. Histogram $\{4:6, 3:187, 2:377\}$ is asserted.
- **auto** (anything else): $n_{\text{contacts}} \geq 2 \rightarrow \text{gap } 4 \text{ iff } d \geq 3.0 \text{ and } b \geq 3 \text{ else } 3;$
 $n_{\text{contacts}} = 1 \rightarrow \text{gap } 3 \text{ iff charged-or-} d \geq 2.5 \text{ else } 2; n_{\text{contacts}} = 0 \rightarrow 2.$

1.3 Function inventory (all demonstrated, run 10)

Function	Role
<code>tuple_of(aa)</code>	$aa \rightarrow$ 12-slot glyph tuple
<code>glyph_distance(t1,t2)</code>	Euclidean ordinal distance
<code>bottleneck_count(t1,t2)</code>	7-slot disagreement count
<code>is_charged_glyph(aa)</code>	D/E/K/R test (B4 split-stratum)
<code>counter_engagement_conf(cdr3, counter)</code>	canonical 0.962, else 7-bottleneck mean agreement
<code>gap_of_pinned(pos, wt, mut)</code>	CORE4/TIER3/BASE gap
<code>gap_of_auto(pos, wt, mut, n_contacts)</code>	glyph-routed gap
<code>choose_wt_codons(wt)</code>	per-residue codon maximizing synonymous-neighbourhood
<code>_neighbors(codon)</code>	single-nucleotide neighbourhood of a codon
<code>translate(dna)</code>	codon-table back-translation
<code>parse_index_set(s)</code>	"0,2,4,6-8" $\rightarrow \{0,2,4,6,7,8\}$; None $\rightarrow$ None
<code>read_wt_from_file(path)</code>	FASTA/plain-text reader
<code>autodetect_contacts(n)</code>	even $\rightarrow$ CDR1, odd $\rightarrow$ CDR2, every-3rd $\rightarrow$ CDR3

Function	Role
<code>contact_count(pos, contacts)</code>	how many CDRs touch pos
<code>build_landscape(wt, codons, gap_fn)</code>	all ($n \cdot 19$) single-aa mutants, gap + codon-viability
<code>frob_of(m)</code>	gap ≥ 3
<code>render_top(top, n)</code>	WT/MUT alignment + caret (60-aa window)
<code>parse_args / main</code>	CLI + 8-step pipeline

2 CLI runs — captured outputs

2.1 01 — Default run (canonical WT, pinned gap model) — exit 0

`python3 hrace_driver.py`

```
hiv_arms_race_driver — full build (30-aa WT, 0-based, gap=pinned)
[1] landscape: 570 single-aa mutants in 1 ms
[2] gap histogram: {4: 6, 3: 187, 2: 377}
[3] codon-viable: 180/570 (low-risk escapes)
[4] escape threats (gap>=3, frob=True): 193
[5] TOP ESCAPE (0-based): pos 15 WT[15]= to D gap=4 frob=True conf
    =0.80
    WT : TGPCTNVSTVQCTHGIRPVVSTQLLLLNGSL
    MUT: TGPCTNVSTVQCTHGDRPVVSTQLLLLNGSL
           ^ pos 15 (0-based mutation site)
[6] COUNTER CDR3: WGENSITKPAVGS engages >T≡I... (full VH Frobenius-OK
    conf 0.962, threshold 0.962)
[7] VECTOR: 29 aa payload -> 87 nt DNA, round-trip OK
    DNA:
    AATGGTATTTCTCATACTAAACCTGCTGTTGGTTCTGGTGGTGGTGGTTCTTGGGGTAATTCTATTACTAAACCTGCTG

[8] COVERAGE: 193/193 gap>=3 mutants retain a CDR1-or-CDR2 contact —
    100%
FROBENIUS: all checks closed (B4=T). Driver: imgct/Vox/hrace_driver.py
(30-aa WT, 0-based, gap=pinned)
```

All internal checksums held: histogram {4:6, 3:187, 2:377} == CANONICAL_HIST, gap-4 pins == the six CORE4 pins, top escape == (15, I→D, gap 4), mut_seq TGPCTNVSTVQCTHGDRPVVSTQLLLLNGSL, coverage 193/193.

2.2 05 — --top-conf 0.60 (canonical) — exit 0

```
python3 hrace_driver.py --top-conf 0.60
```

Identical full 8-step pipeline (pinned is forced for canonical); the lowered threshold widens the eligible set but the top is unchanged: pos 15 I→D, gap 4, conf 0.80. Shows the threshold gates *eligibility*, not the winner, when the canonical pin is special-cased at conf 0.80.

2.3 06 — --counter-conf 0.99 (canonical) — exit 0

```
python3 hrace_driver.py --counter-conf 0.99
```

Step[6] flips to: WGN~~S~~ITKPAVGS does NOT engage >·T·~~W~~·~~I~~ (conf 0.962, threshold 0.990). The engagement verdict is a threshold comparison on the same computed conf — raising the bar above the canonical 0.962 demotes the canonical counter from “engages” to “does NOT engage” with no other step changing.

2.4 07 — Custom counter CDR3 — exit 0

```
python3 hrace_driver.py --counter-cdr3 AAAAAAAAAAAAAAAAAA
```

Step[6]: AAAAAAAAAAAAAAAAAA does NOT engage ... (conf 0.512, threshold 0.962) — computed via the 7-bottleneck mean-agreement formula (not the canonical shortcut). Step [7] payload grows to 33 aa → 99 nt DNA, round-trip still OK.

2.5 17 — Custom WT, auto mode, --top-conf 0.55 — exit 0, full 8 steps

```
python3 hrace_driver.py --wt AGLTVWPSQNRDEHKCITFVYA --top-conf 0.55
```

```
hiv_arms_race_driver — full build (22-aa WT, 0-based, gap=auto)
[1] landscape: 418 single-aa mutants in 3 ms
[2] gap histogram: {4: 86, 3: 296, 2: 36}
[3] codon-viable: 134/418 (low-risk escapes)
[4] escape threats (gap>=3, frob=True): 382
[5] TOP ESCAPE (0-based): pos 0  WT[0]= to R  gap=4  frob=True  conf=0.60
    WT : AGLTVWPSQNRDEHKCITFVYA
    MUT: RGLTVWPSQNRDEHKCITFVYA
        ^ pos 0 (0-based mutation site)
[6] COUNTER CDR3: WGNSITKPAVGS engages >TW·I··· (conf 0.962, threshold
    0.962)
[7] VECTOR: 29 aa payload -> 87 nt DNA, round-trip OK
[8] COVERAGE: 382/382 gap>=3 mutants retain a CDR1-or-CDR2 contact —
    100%
FROBENIUS: all checks closed (B4=T).
```

Auto mode end-to-end: autodetected contacts (even→CDR1, odd→CDR2, every-3rd→CDR3), glyph-routed gaps, top escape at pos 0 (A→R, gap 4 conf 0.60, tie-broken to lowest pos among gap-4 conf-0.60 candidates), 100% coverage by construction of autodetection (every position is in CDR1∪CDR2).

2.6 16 — --help — exit 0

Full option table: --wt, --wt-file, --cdr1/2/3, --counter-cdr3, --linker, --cdr1-contacts/--cdr2-contacts/--cdr3-contacts (comma/range lists), --gap-model {pinned,auto}, --counter-conf, --top-conf.

2.7 02 — Custom WT, default thresholds — exit 2 (eligibility gate)

```
python3 hrace_driver.py --wt AGLTVWPSQNRDEHKCITFVYA
```

```
[1] landscape: 418 single-aa mutants in 3 ms
[2] gap histogram: {4: 86, 3: 296, 2: 36}
[3] codon-viable: 134/418
[4] escape threats (gap>=3): 382
error: no escape candidate meets --top-conf 0.80      (exit 2)
```

The pipeline runs steps [1]–[4] and then dies at the TOP ESCAPE gate: in auto mode the conf formula is 0.60 for gap-4 and $0.45 + 0.001 \cdot \text{pos}$ otherwise ($\leq \sim 0.47$ for 22-aa), so the default --top-conf 0.80 — calibrated for the canonical special-cased 0.80 pin — is unreachable by any non-canonical input. (Step [5] onward, including all 8 steps, completes once the threshold is lowered: see run 17.)

2.8 08 — --wt-file FASTA input — exit 2, same gate

```
printf '>custom ep\textbackslash nAGLTVWPSQNRDEHKCITFVYA\textbackslash n' > /tmp/hrace_wt.fa && python3 hrace_driver.py --wt-file /tmp/hrace_wt.
```

FASTA/plain-text reader works (22-aa WT parsed, >header stripped, identical landscape to run 02: 418 mutants, same histogram) and hits the same top-conf gate. Demonstrates the file-input lane end-to-end.

2.9 09 — Short 10-aa WT — exit 2, same gate; also exercises the n \$ \le \$ W render path

```
python3 hrace_driver.py --wt AGLTVWPSQNRDEHKCITFVYA --top-conf 0.80
→ 10-aa variant
python3 hrace_driver.py --wt AGLTVWPSQN
```

```
[1] landscape: 190 single-aa mutants in 2 ms
[2] gap histogram: {4: 47, 3: 126, 2: 17}
[3] codon-viable: 59/190
[4] escape threats: 173
error: no escape candidate meets --top-conf 0.80      (exit 2)
```

2.10 03 — Canonical WT + contact overrides (forces auto) — exit 1, assertion crash

```
python3 hrace_driver.py --cdr1-contacts 0,2,4,6 --cdr2-contacts 1,3,5
--cdr3-contacts 2,13,29
```

```
[5] TOP ESCAPE (0-based): pos 15 WT[15]= to D gap=2 frob=True conf
    =0.80
Traceback (most recent call last):
  File ".../hrace_driver.py", line 395, in main
    assert (top['pos'], top['new'], top['gap']) == (15, 'D', 4), top
AssertionError: {... 'gap': 2 ...}
```

With sparse overrides, pos 15 has 0 contacts, so `gap_of_auto` returns 2 for I→D (neither charged-gated at 0 contacts nor $d \geq 2.5$ at $n = 0$). The canonical convention guard (line 395) fires because it is gated on `is_canonical` and `top_conf <= 0.80` — it does not know the gap model is auto, and it demands the pinned gap-4 pin.

2.11 04 — Canonical WT, explicit --gap-model auto, no overrides — exit 1, same crash

```
python3 hrace_driver.py --gap-model auto
```

```
[2] gap histogram: {4: 135, 3: 373, 2: 62}
[5] TOP ESCAPE (0-based): pos 15 WT[15]= to D gap=3 frob=True conf
    =0.80
AssertionError: (line 395, same guard; top gap is 3, not 4)
```

Under canonical contacts, pos 15 has exactly 1 contact (CDR1), and I→D is a charged glyph, so auto gives gap 3. Same guard, same crash — one step earlier in the gap value.

2.12 19 — Canonical auto + --top-conf 0.60 — exit 1; the assert can even catch a *different* mutant

```
python3 hrace_driver.py --gap-model auto --top-conf 0.60
```

```
[5] TOP ESCAPE (0-based): pos 2 WT[2]= to A gap=4 frob=True conf=0.60
AssertionError: {... 'pos': 2, 'new': 'A', 'gap': 4 ...} (line 395)
```

At threshold 0.60 the eligible set includes all gap-4 mutants (conf 0.60); the key (gap, conf, -pos) tie-breaks to the lowest pos among them (pos 2, P→A). So the canonical guard is not just gap-mode blind — it assumes the *identity* of the winner, which auto mode does not preserve.

2.13 18 — Fully non-canonical end-to-end attempt — exit 2 (no gap-4 exists under those contacts)

```
python3 hrace_driver.py --wt AGLTVWPSQNRDEHKCITFVYA --cdr3 NPGAVG --counter
cdr3 GPGGSG --top-conf 0.50 --counter-conf 0.50 --cdr1-contacts 0,3,6,9
--cdr2-contacts 1,4,7,10 --cdr3-contacts 2,5,8,11,14,17,20
```

```
[2] gap histogram: {3: 242, 2: 176}
[4] escape threats: 242
error: no escape candidate meets --top-conf 0.50 (exit 2)
```

These contact sets give no position ≥ 2 contacts with a large enough glyph move, so the histogram has no gap-4 tier; every conf is $0.45 + 0.001 \cdot \text{pos}$ (≤ 0.471) < 0.50 . The gate is doing real work: the landscape built, the histogram is internally consistent ($242 + 176 = 418$), and the refusal is honest.

3 Direct function suite (run 10) — import-level demonstration

Every non-CLI function exercised directly with concrete values:

```
tuple_of('I'): 1= 2= 3= 4= 5= 6= 7= 8= 9= 10= 11= 12= 13= 14= 15= 16= 17= 18= 19= 20=
glyph_distance(I,D) = 3.4641 bottleneck_count(I,D) = 3
glyph_distance(A,K) = 4.2426 bottleneck_count(A,K) = 5
is_charged_glyph: D=True K=True R=True I=False G=False
gap_of_pinned(15,I,D) = 4 (the kernel-verified →ID escape pin)
gap_of_pinned(15,I,E) = 3 (CORE4, not a pin → 3)
gap_of_pinned(0,T,D) = 3 (TIER3, charged mutant → 3)
gap_of_pinned(14,G,E) = 2 (BASE → always 2)
gap_of_auto(2 contacts, →ID) = 4 (d=3.4641 ≥ 3.0, b=3 ≥ 3)
gap_of_auto(1 contact, →IE) = 3 (charged-glyph gate)
gap_of_auto(0 contacts, →ID) = 2 (no contact → floor)
counter_engagement_conf(canonical) = 0.962 (hard-coded pin)
counter_engagement_conf(canonical CDR3 vs AAAAAAAAAAAAAA) = 0.5119 (
computed)
parse_index_set('0,2,4,6-8') = {0,2,4,6,7,8}; parse_index_set(None) =
None
```

```

autodetect_contacts(10) = CDR1 {0,2,4,6,8}, CDR2 {1,3,5,7,9}, CDR3
    {0,3,6,9}
contact_count(2, canonical) = 2    (pos 2 in CDR1 and CDR3)
choose_wt_codons('GIRP') = ['GGT', 'ATT', 'CGT', 'CCT']    (max synonymous
    neighbourhood)
_neighbors('ATT') = {F, I, L, M, N, S, T, V}    (8 single-nt
    neighbours)
vector: len(CDR3+linker+counter)=29 → translate(dna)==payload: True
canonical WT[15] = I    (the →ID escape site)

```

Notes:

- `choose_wt_codons` picks, per residue, the codon with the largest synonymous single-nucleotide neighbourhood — e.g. ATT for Ile, whose neighbourhood includes the Met (ATG) and Lys/Asn-class codons. This is what the **codon-viable** count measures: a mutant is “viable” iff its residue is one single nt away from the chosen WT codon.
- `tuple_of` is total over the 20 standard aa and raises `ValueError` on anything else (exercises the AA-string domain guard, also enforced by `main` before the pipeline).

4 Error paths — all six validation lanes, exit 2

#	Command (abridged)	Output	Guard
11	<code>--wt ...X...</code> (non-standard residue)	error: non-standard residue(s) in WT: ['X']	WT domain check, pre-pipeline
12	<code>--counter-cdr3</code> <code>NGISHTKPAVGS</code> (= <code>cdr3</code>)	error: <code>--counter-cdr3</code> must differ from <code>--cdr3</code>	payload sanity, pre-pipeline
13	non-canonical WT + <code>--gap-model</code> pinned	error: <code>--gap-model</code> pinned requires the canonical WT/CDRs and no contact overrides	mode/entry consistency
14	<code>--cdr1-contacts 99</code> (22-aa WT)	error: CDR1 contact index 99 out of range 0..21	contact range, per CDR

#	Command (abridged)	Output	Guard
15	canonical + --top-conf 0.95	error: no escape candidate meets --top-conf 0.95	early canonical conf check (line 310) — fires before any pipeline output, since nothing above 0.80 exists in the pinned run
—	--wt '' (empty)	error: empty WT sequence (line 291)	empty-input guard

All refusals are clean: one line to stderr, exit 2, no partial pipeline output (except where the gate sits after steps [1]–[4], which is by design — see §6.3).

5 What the results show

The driver is a checksummed, self-certifying pipeline, and the canonical run is its fixed point. Run 01 closes all eight steps with every internal assert holding: the gap histogram {4:6, 3:187, 2:377} is not just reported but *asserted* equal to CANONICAL_HIST, the six gap-4 mutants are asserted to be exactly the kernel-verified CORE4 pins, the top escape is asserted to be pos-15 I→D at gap 4 with the exact mutated sequence, and the coverage is asserted 193/193. The final line FROBENIUS: all checks closed (B4=T) is therefore not a slogan — on this run it is backed by six executed assertions. The glyph algebra is doing the scoring: I→D reads distance 3.4641 with 3 bottleneck disagreements, which is precisely the shape gap_of_auto would also promote to gap 4 when two or more contacts are present — the pinned table and the glyph rule agree on the canonical escape, which is the point of the pin.

The two gap models are the same algebra read two ways. Pinned reads the position class from the kernel-verified partition and only the charged-glyph test inside TIER3; auto reads *both* class and severity off the glyph tuple plus the local contact count. Run 04 shows auto recovering most of the pinned shape on its own (135 gap-4 mutants vs 6 pinned — auto over-assigns gap 4 to high-movement mutants at ≥ 2 contacts, the pinned table withholds), and run 10 shows the boundary cases of each rule in isolation: CORE4→4 only for the six pins, TIER3→3 iff charged, BASE→2 always; auto→4 iff $d \geq 3$ and $b \geq 3$ at ≥ 2 contacts, →3 iff charged-or- $d \geq 2.5$ at 1 contact, →2 otherwise.

The thresholds are real gates, not formatting. --top-conf changes *who is eligible* (run 05: widens eligibility, same winner) and run 19 shows it can even change the *identity* of the winner once gap-4 mutants at conf 0.60

enter the pool (top becomes pos-2 P→A). `--counter-conf` moves the engagement verdict between “engages” and “does NOT engage” on an unchanged computed conf (runs 01/06), and a non-canonical counter gets a *computed* 7-bottleneck conf (run 07: 0.512).

The vector build is an exact codec, not a lookup. Step [7] assembles CDR3+linker+counter, encodes with the first codon of each residue, and asserts `translate(dna) == payload` — 29 aa → 87 nt canonically, 33 aa → 99 nt with the custom counter, both round-tripping. The codon-viable count (180/570 canonical, 134/418 custom) is the same machinery on the WT side: a mutant is low-risk iff its residue sits in the single-nucleotide neighbourhood of the max-neighbourhood WT codon (run 10: `_neighbors('ATT') = 8` residues).

Every input lane works. `--wt` (01, 02, 17), `--wt-file` FASTA with header stripping (08), contact overrides with ranges (03), explicit gap model (04, 19), short sequences through the non-windowed render path (09), and all six validation refusals with clean exit-2 semantics (11-15, empty-WT). `--help` documents the full surface (16).

6 Findings — where the suite exposes tension

1. **The default `--top-conf 0.80` is canonical-only.** Auto-mode confs top out at 0.60 (gap-4) and $0.45 + 0.001 \cdot \text{pos}$ otherwise, so *every* non-canonical run with default thresholds dies at the eligibility gate (runs 02, 08, 09, 18). The 0.80 value is the special-cased canonical pin; it is not reachable anywhere else. Non-canonical use requires `--top-conf $ \le $ 0.55-ish` — the `--help` text does not say this.
2. **The canonical convention guard (line 395) is gap-mode blind.** It fires on `is_canonical` and `top_conf $ \le $ 0.80` and demands the top be exactly (15, 'D', 4) — but under auto gaps the top is (15, 'D', 3) (run 04), (15, 'D', 2) (run 03), or even (2, 'A', 4) (run 19). Result: **canonical WT + auto gap can never complete** — `top_conf $ \le $ 0.80` crashes with `AssertionError` (exit 1), and `top_conf > 0.80` is rejected early (line 310). The one-line fix is to add `and gap_mode == 'pinned'` to the guard’s condition; as written, `--gap-model auto` on the canonical WT is a dead end.
3. **Coverage 100% under auto is by construction, not by measurement.** `autodetect_contacts` puts every even position in CDR1 and every odd in CDR2, so `CDR1 ∪ CDR2` is the whole epitope and step [8] cannot report below 100% for autodetected inputs (runs 09, 17). It is only informative on canonical (193/193, where the contacts are real) or explicit-override inputs.
4. **`stderr/stdout` interleave is an artifact, not a bug.** In runs 02/08/09 the error line appears *before* the pipeline header in captured output; the

gate at line 383 executes after steps [1]-[4] — unbuffered stderr simply overtakes buffered stdout under `2>\&1`.

7 Reproduce

```
cd /home/mrnob0dy666/imsqct/Vox
python3 hrace_driver.py                                # 01
    canonical pinned
python3 hrace_driver.py --wt AGLTVWPSQNRDEHKCITFVYA --top-conf 0.55 #
    17 auto, full 8 steps
python3 hrace_driver.py --counter-conf 0.99            # 06
    engagement demoted
python3 hrace_driver.py --counter-cdr3 AAAAAAAAAAAAAAA # 07 computed
    conf
python3 hrace_driver.py --wt-file my.fa                # 08 FASTA
    lane
python3 hrace_driver.py --gap-model auto               # 04 guard
    crash (by design, pre-fix)
```

Full transcripts: /tmp/hrace_results.txt, /tmp/hrace_results2.txt.

8 BUGS FOUND & FIXED

Two real defects in the shipped driver; both fixed in place (pre-fix copy: `hrace_driver.py`). No behavior change on any previously working input — the canonical default run is byte-identical to the pre-fix transcript.

8.1 Convention guard was gap-mode blind (crash, exit 1)

The post-pipeline guard

```
if is_canonical and args.top_conf <= CANONICAL_TOP_CONF:
    assert (top['pos'], top['new'], top['gap']) == (15, 'D', 4), top
```

assumed the canonical pinned outcome under *any* configuration. With `--gap-model auto` (runs 04, 19) or a contact override forcing auto (run 03) the legitimate auto-mode winner differs, so the driver printed a correct TOP ESCAPE and then died on its own assert. Fix: the guard now fires only under the model it encodes —

```
if gap_mode == 'pinned' and is_canonical and top_conf <=
    CANONICAL_TOP_CONF:
```

The pinned-mode pinned assert (histogram, pins, winner, mut_seq, coverage 193) still executes on every canonical pinned run; it is simply no longer applied to a model it does not describe.

8.2 Default --top-conf 0.80 was unreachable off the canonical special case

Confidence is 0.80 only for the canonical (15, D) pair; the gap-4 tier is 0.60 and everything else $0.45 + 0.001 \cdot \text{pos}$. With the hard default 0.80, every non-canonical or auto run (runs 02, 08, 09, 18) exited 2 before finishing — the default made the tool’s own auto mode unusable out of the box. Fix: --top-conf now defaults to None and resolves after the gap mode is known:

```
top_conf = args.top_conf if args.top_conf is not None else (  
    CANONICAL_TOP_CONF if gap_mode == 'pinned' else 0.60)
```

Canonical pinned runs still default to 0.80; auto runs default to the gap-4 tier (0.60), which is the highest tier the auto gate can award by construction. The early reject (line 313) and the eligibility gate (line 384) now use the resolved value, and the early reject applies only in pinned mode — a canonical auto run with --top-conf 0.95 now reports a clean typed exit 2 from the eligibility gate instead of skipping the whole pipeline.

8.3 Regression matrix (post-fix)

case	before	after
01 canonical default	exit 0	exit 0, byte-identical output
02 custom 22-aa WT, default conf	exit 2	exit 0, top pos0 A→R gap4 conf 0.60
03 canonical + CDR3 contact override	exit 1 (assert)	exit 0, top pos15 I→D gap3 conf 0.80
04 canonical --gap-model auto	exit 1 (assert)	exit 0, top pos2 P→A gap4 conf 0.60
05 canonical --top-conf 0.60	exit 0	exit 0, unchanged
06 canonical --counter-conf 0.99	exit 0	exit 0, unchanged
15 canonical --top-conf 0.95	exit 2	exit 2, same early reject
17 custom 22-aa + --top-conf 0.55	exit 0	exit 0, unchanged
19 canonical auto --top-conf 0.60	exit 1 (assert)	exit 0, top pos2 P→A gap4 conf 0.60
11 non-standard residue X	exit 2	exit 2, unchanged
12 counter == cdr3	exit 2	exit 2, unchanged
13 pinned on non-canonical	exit 2	exit 2, unchanged

case	before	after
14 contact index 99	exit 2	exit 2, unchanged
empty WT	exit 2	exit 2, unchanged
NEW canonical auto --top-conf 0.95	(skipped pipe)	exit 2, clean eligibility gate
NEW canonical over- ride + explicit pinned	exit 2	exit 2, unchanged

What the fixes show: the driver’s convention asserts were written *as* the canonical pinned derivation — correct, but they were being enforced on inputs the derivation does not cover. Pinning each assertion to the model it encodes (`gap_mode == 'pinned'`) and resolving the threshold default per model is the minimal change that keeps every previously green run byte-identical while making the auto mode reachable by default. The 100% auto-mode coverage figure (run 02: 382/382) is, as noted in §6, a by-construction artifact of `autodetect_contacts` and is unaffected by these fixes.